

Natural Language Processing

Project Documentation

1. Student Information

Full Name	Hackers
GitHub	https://github.com/mohamedhosne/SMS-Spam-Detection.git
Department / Major	Computer science
Academic Year	Level - 4
Group Members	Eng / : MOHAMED HOSNY . Eng / : OMAR MOHAMED. Eng / : MOSTAFA MAHMOUD. Eng / : MOHAMED EHAB. Eng / : MOUSA WAGDY.

2. Project Overview

Project Title	SMS Spam Detection Using Traditional Machine Learning and Hybrid CNN-LSTM Architecture
Submission Date	May 29, 2026

Project Summary

This project focuses on building an automated, high-performance SMS Spam Detection system to counter security and privacy threats in mobile communication. Utilizing the benchmark UCI SMS Spam Collection dataset, the system implements a comprehensive Natural Language Processing (NLP) pipeline, including text cleaning with Regular Expressions (Regex) and text normalization via NLTK (stopwords removal and lemmatization). The project benchmarks traditional Machine Learning approaches (Decision Tree and XGBoost) utilizing TF-IDF vectorization against an advanced Hybrid Deep Learning architecture combining Convolutional Neural Networks (CNN) for local spatial feature extraction and Long Short-Term Memory (LSTM) networks for temporal sequence tracking. The models are evaluated using metrics such as Accuracy, Precision, Recall, and F1-Score, aiming to deliver a robust and scalable solution that effectively minimizes false positives and ensures secure message filtering.

3. NLP Topic Area

Select the NLP topic(s) your project will focus on:

- | | | |
|---|---|---|
| ☒ Text Classification | ☐ Sentiment Analysis | ☐ Named Entity Recognition |
| ☐ Machine Translation | ☐ Text Generation / LLMs | ☐ Speech Processing |
| ☐ Question Answering | ☐ Text Summarization | ☐ Other (specify below) |

If 'Other', specify the NLP topic:

Text Classification

4. Problem Statement

Problem Statement

With the massive increase in mobile communication, SMS spam has become a critical security and privacy threat, leading to financial fraud, phishing scams, and an degraded user experience. Traditional rule-based filtering methods fail to adapt to the evolving and sophisticated patterns used by spammers.

This project aims to address this issue by developing an automated, high-performance SMS Spam Detection system using the UCI SMS Spam Collection dataset. The objective is to evaluate and compare traditional Machine Learning algorithms (such as Decision Trees and XGBoost) utilizing TF-IDF vectorization, against an advanced Deep Learning hybrid architecture combining Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks. The ultimate goal is to build a robust classifier capable of accurately distinguishing between legitimate messages (ham) and unsolicited advertisements (spam) while minimizing false positives to ensure reliable message delivery.

Motivation & Relevance

Short Message Service (SMS) remains a universally used communication channel, making it a primary target for malicious actors. Evaluating advanced machine learning and hybrid deep learning models is highly relevant to developing modern, real-time firewalls for mobile devices. This research provides deep insights into how lexical patterns, spatial feature extraction (CNN), and long-term sequential dependencies (LSTM) contribute to understanding and combating modern spam techniques, which directly enhances cybersecurity measures in telecommunications.

5. Proposed Methodology

Programming Language

Python (v3.10)

Frameworks / Libraries

TensorFlow, Keras, Scikit-Learn, XGBoost, Pandas, NumPy, NLTK, Regex

Dataset / Link

UCI SMS Spam Collection Dataset
(<https://archive.ics.uci.edu/static/public/228/sms+spam+collection.zip>)

Data Collection & Preprocessing

Data Collection: The dataset is fetched programmatically from the official UCI Machine Learning Repository, containing 5,574 labeled SMS messages.

Text Cleaning: Raw text is standardized to lowercase. Regular Expressions (Regex) are utilized to replace URLs, financial symbols, and digit sequences with specific placeholders (URL, MONEY, NUMBER) to minimize feature noise.

Tokenization & Stopwords: Text is split into individual words where standard English stopwords are filtered out via NLTK.

Lemmatization: Words are reduced to their dictionary base form using the WordNetLemmatizer.

Data Splitting: The refined data is mapped to binary targets (0 for ham, 1 for spam) and split into 80% training and 20% testing sets using stratified sampling to maintain class proportions.

Model / Algorithm Approach

This project builds and compares two analytical approaches:

Traditional Machine Learning: Uses TF-IDF Vectorization (max 3,000 features) combined with a Decision Tree Classifier (max_depth=20) and an optimized XGBoost Classifier using ensemble learning.

Hybrid Deep Learning (CNN + LSTM): Words are processed into sequences (maxlen=50). A **Conv1D** layer extracts local n-gram text patterns, a **MaxPooling1D** downsamples the features, and an **LSTM** layer captures long-term sequential context. The network is regularized using Dropout (0.3 & 0.5), L2, and early stopping.

Evaluation Metrics

The models are evaluated on the testing dataset using Scikit-Learn's classification_report with the following metrics:

- **Accuracy Score:** To track overall performance on training and testing splits.
- **Precision:** Crucial for the Spam class to reduce false positives, ensuring safe messages (Ham) aren't blocked.
- **Recall (Sensitivity):** Measures the ability to catch all actual spam.
- **F1-Score:** The harmonic mean of precision and recall, serving as the main indicator due to class imbalance.

6. Project Framework

- **Data Input Layer:** Programmatic ingestion of the raw UCI SMS Spam Collection dataset.
- **Preprocessing Module:** Text normalization using Regex (masking URLs, numbers, and currencies) and NLTK (stopwords removal and lemmatization).
- **Feature Representation Module:** Dual-stream processing using TF-IDF Vectorization for Machine Learning, and Sequence Tokenization with Post-Padding for Deep Learning.
- **Model Training Architecture:** Benchmarking traditional Machine Learning baselines (Decision Tree, XGBoost) against a sequential Hybrid Deep Learning pipeline (Conv1D + MaxPooling1D + LSTM).
- **Evaluation Module:** Performance assessment and verification using validation splits and Scikit-Learn metrics (Accuracy, Precision, Recall, and F1-Score).

7. Expected Challenges & Mitigation

Anticipated Challenges

- ❑ **Class Imbalance:** The dataset contains significantly more legitimate messages (Ham) than spam, which can bias the models.
- ❑ **Overfitting Risk:** High potential for the complex Hybrid Deep Learning model (CNN + LSTM) to overfit on the limited text dataset.
- ❑ **Hardware & Memory Constraints:** Large embedding layers and deep learning structures can lead to high RAM usage and slow CPU training on native Windows environments.

Mitigation Plan

- ❑ **Stratified Splitting & Metrics:** Using stratify=y during data splitting to maintain class proportions, and evaluating performance using F1-Score instead of just accuracy.
- ❑ **Regularization Techniques:** Incorporating L_2 regularization, Dropout layers (0.3 and 0.5), and EarlyStopping to halt training before overfitting occurs.
- ❑ **Resource Optimization:** Restricting vocabulary size (max_features=3000), optimizing batch sizes, and clearing local pip cache to manage memory limits.

8. Preliminary References

1. Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., ... & Zheng, X. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org.
2. Almeida, T. A., Hidalgo, J. M. G., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. *Proceedings of the 11th ACM Symposium on Document Engineering*, 259–262.
3. Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python: Analyzing text with the natural language toolkit*. O'Reilly Media, Inc.
4. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
5. Chollet, F. (2015). Keras. GitHub. Retrieved from <https://github.com/fchollet/keras>
6. Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362.

-
7. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.
 8. Kim, Y. (2014). Convolutional neural networks for sentence classification. *arXiv preprint arXiv:1408.5882*.
 9. McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 445, 51-56.
 10. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(10), 2825-2830.
 11. Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81-106.
 12. Roy, P. K., Asif, J. I., Islam, A. S., & Islam, M. M. (2020). A hybrid CNN-LSTM model for SMS spam detection. *IEEE International Conference on Computer and Information Technology*, 1-6.
 13. UCI Machine Learning Repository. (2012). SMS Spam Collection Dataset. Retrieved from <https://archive.ics.uci.edu/ml/datasets/SMS+Spam+Collection>
 14. Wu, J., & Zhou, L. (2022). Deep learning-based SMS spam detection: A comparative study of CNN and LSTM. *Journal of Cyber Security and Mobility*, 11(3), 415-432.
 15. Zhou, C., Sun, C., Liu, Z., & Lau, F. (2015). A C-LSTM neural network for text classification. *arXiv preprint arXiv:1511.08630*.